OpenArm

Gesture-Controlled Robotic Arm

Google Drive link: Output Video Folder

Contents

1 Introduction **1**

2 Background and Literature Review **2**

3 Problem Statement **3**

4 System Overview **4**

5 Hardware Design **6**

5.1 Hardware Design — Mechanical, Actuation and Electronics 6

5.1.1 Mechanical Structure (Links, Joints, Materials) 6

5.1.2 Actuation (Servos and Motors) 7

5.1.3 Electronics (MCU, IMU, Power) 7

5.1.4 CAD Models Details 8

5.1.5 Material Selection & Justification 9

5.1.6 CAD Model — Part Names and Roles 10

6 Software **11**

6.0.1 System Overview 11

6.0.2 Hand Tracking and Palm Center Estimation 11

6.0.3 Depth Estimation via Palm Width 12

6.0.4 Workspace Mapping and Constraints 13

6.0.5 Gripper Control via Gesture Recognition 13

6.0.6 Communication Protocol 13

6.0.7 Integration with CoppeliaSim 14

6.0.8 User Interaction 14

6.0.9 Key Features 14

7 Conclusion **15**

1 Introduction

OpenArm is a low-cost, gesture-controlled robotic arm built from scratch to demonstrate intuitive human–robot interaction. Instead of using buttons or joysticks, the system uses wearable IMU-based gesture sensing, allowing the user's hand motions to directly control the arm in real time.

Gesture-controlled manipulators are increasingly relevant because they offer natural, hands-free, and unobtrusive control, making robotic systems easier to use in environments where cameras, tracking systems, or complex interfaces are impractical. Such systems are valuable in assistive robotics, teleoperation, education, and interactive demonstrations.

The motivation of OpenArm is to create an accessible platform that combines simple hardware, low-cost sensors, and real-time motion mapping. The project goals are to design a functional robotic arm, develop a wearable gesture interface, implement stable gesture-to-joint mapping, and demonstrate basic manipulation tasks.

2 Background and Literature Review

Robot manipulators are mechanical systems consisting of rigid links connected by joints, designed to position and orient an end-effector in 3D space. Their behavior is modeled using kinematics: forward kinematics computes the end-effector pose from joint angles, while inverse kinematics determines the required joint angles for a desired position. These models enable stable motion planning and control in multi-DOF robotic arms.

Gesture recognition systems typically rely on sensors that capture human motion patterns. IMU-based methods use accelerometers, gyroscopes, and magnetometers to track hand orientation and dynamic gestures without requiring external infrastructure. Vision-based systems detect gestures using cameras and deep learning but are sensitive to lighting and occlusion. EMG-based systems measure muscle activation to decode hand intent and offer fine-grained control but require skin contact and careful calibration.

Related work in wearable control and assistive robotics shows increasing interest in natural human–robot interfaces. Wearable IMU gloves, armbands, and exoskeleton-style sensors have been used to teleoperate robotic arms for rehabilitation, remote manipulation, and human–robot collaboration. Prior research highlights the advantages of low-cost inertial sensing, reduced user training, and improved accessibility, motivating lightweight gesture-driven platforms like OpenArm.

3 Problem Statement

There is a growing need for intuitive control methods that allow users to operate robotic arms naturally and efficiently. Gesture-controlled interfaces provide a direct and human-friendly way to command a robot without joysticks, keyboards, or complex programming, making them ideal for education, remote manipulation, and assistive applications.

Most existing low-cost robotic arms either rely on manual buttons or predefined motion sequences, offering limited flexibility and poor user experience. They lack real-time, fluid, and intuitive control, and often depend on bulky or proprietary hardware. As a result, they are not suitable for gesture-driven interaction or rapid prototyping of human–robot control systems.

The objectives of this project are:

Hardware: Design and build a lightweight robotic arm using affordable components and reliable actuation.

Software: Develop firmware and data-processing pipelines for gesture capture and communication.

Control: Implement real-time gesture-to-joint mapping using IMU data with smooth and stable motion control.

Testing: Evaluate the system for responsiveness, accuracy, and usability across basic manipulation tasks.

4 System Overview

A high-level block diagram.

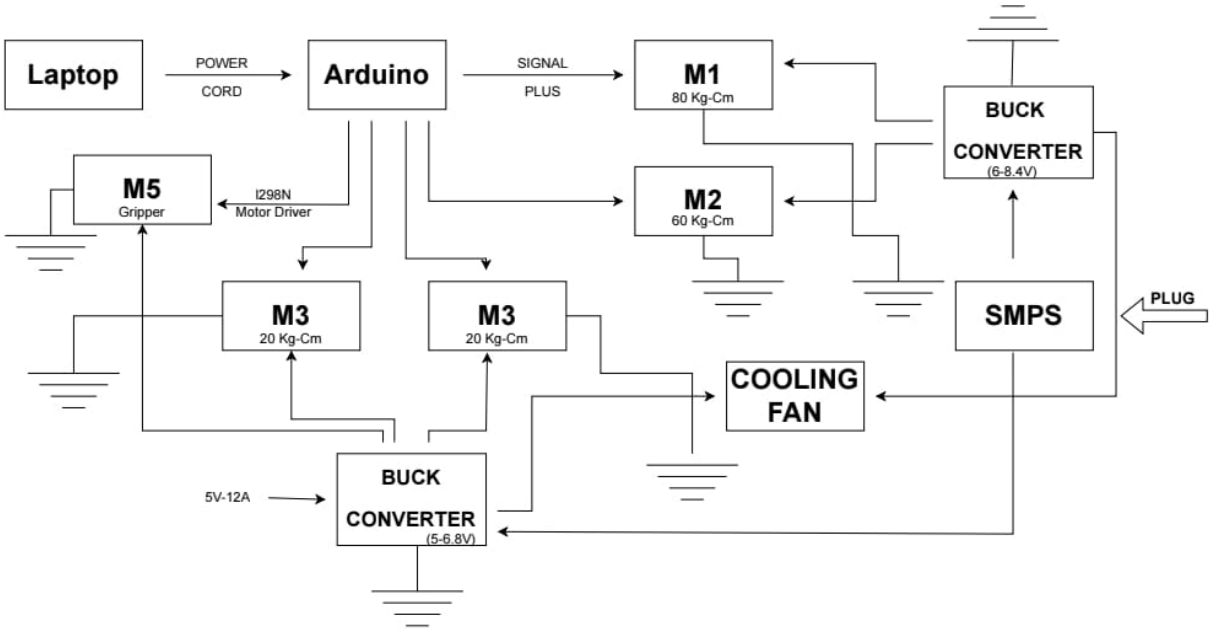

Figure 4.1: Overall system architecture of the OpenArm platform.

Block-wise Description

Laptop: Acts as the programming and monitoring interface. It is used to upload code, visualize serial data, and communicate with the Arduino. The laptop also supplies power to the Arduino through the USB connection when required.

Arduino: Serves as the central controller of the system. It processes gesture commands and generates PWM signals for all servomotors. It also drives the L298N motor driver for the gripper.

M1 (80 kg-cm Servo): A high-torque servo responsible for base rotation of the arm. It is powered through a dedicated 6–8.4 V buck converter due to its high current demand.

M2 (60 kg-cm Servo): Controls the shoulder joint and requires a stable, high-current supply. It shares the same regulated voltage source as M1.

M3, M3 (20 kg-cm Servos): Two medium-torque servos used for the elbow and wrist joints. They operate at 5–6.8 V and are powered through a secondary buck converter.

M5 (Gripper Motor): Drives the gripper mechanism. Since it is a DC/gear motor, it is controlled through the L298N motor driver rather than directly from the Arduino.

L298N Motor Driver: An H-bridge motor driver used to control the direction and speed of the gripper motor (M5). The Arduino provides control signals, while the motor driver supplies the required power.

SMPS (Switched-Mode Power Supply): The main AC-to-DC power unit for the system. It outputs 12 V at high current, powering the buck converters and cooling fan.

Buck Converter (6–8.4 V): Steps down the SMPS 12 V output to a safe range suitable for powering high-torque servos M1 and M2.

Buck Converter (5–6.8 V): Provides regulated lower voltage for the M3 servos and supporting electronics.

Cooling Fan: Powered directly by the SMPS, it provides airflow to prevent overheating of the buck converters and servomotors during extended operation.

Common Ground: All components share a unified ground connection, ensuring stable signal reference and noise-free servo operation.

5 Hardware Design

5.1 Hardware Design — Mechanical, Actuation and Electronics

5.1.1 Mechanical Structure (Links, Joints, Materials)

The OpenArm mechanical structure follows a conventional serial manipulator layout: a fixed *base_plate*, a *shoulder_link* (primary segment), a *forearm_link* (secondary segment), intermediate *elbow_bracket* joints, a *wrist_rotator*, and the *gripper_finger* end-effector. Links are designed as lightweight hollow members while joint connectors concentrate loads and house bearings/servo outputs.

Design principles:

- **Modularity:** each link and joint is a replaceable module with standardized bolt patterns for easy assembly and maintenance.
- **Stiffness where needed:** high bending stiffness for long links (forearm/shoulder) to limit deflection under payload.
- **Local reinforcement:** joint connectors and servo mounts are reinforced using thicker material or inserts to avoid wear.
- **Compact envelopes:** minimize link cross-section to reduce inertia and improve dynamic response.

Recommended geometric details:

- Shoulder and forearm: hollow tubes (carbon-fibre or thin aluminium) with outer diameter 20–30 mm and wall thickness 1.5–3 mm.

- Joint flanges / brackets: aluminium plates 2–4 mm thick depending on load path.
- Fasteners: M3/M4 for small brackets, M6 for base mounting.
- Bearings: use flanged plain bearings or small deep-groove bearings at rotating interfaces to remove backlash.
- Typical manufacturing tolerance: ±0.2 mm for printed parts, ±0.1 mm for machined/aluminium parts.

5.1.2 Actuation (Servos and Motors)

Actuation is provided by hobby/high-torque servos and a DC gripper motor:

- **Base (M1)** — High-torque servo (specified: 80 kg-cm). Responsible for base rotation; requires steady high-current supply and metal gearbox for durability.
- **Shoulder (M2)** — High/medium torque servo (60 kg-cm). Large moment arm; select servo with metal gears and adequate stall torque.
- **Elbow, Wrist (M3)** — Medium torque servos (20 kg-cm) for mid-span joints and wrist articulation.
- **Gripper (M5)** — DC geared motor controlled via an H-bridge (L298N) for bidirectional open/close control and variable speed.

Control signals are PWM from the MCU. Select servos with metal gears, metal splines on output horns, and metal or reinforced servo mounts to avoid stripping under load. Use external power (buck-regulated) to avoid brownouts on the MCU USB rail.

5.1.3 Electronics (MCU, IMU, Power)

MCU: An Arduino (or similar 8/32-bit MCU) acts as central controller—reads gesture input, runs mapping/filtering and issues PWM/driver commands.

If using multiple IMU streams or advanced sensor fusion, prefer a 32-bit MCU (e.g., ESP32 / STM32) for headroom.

Power Architecture:

- **SMPS (12 V)** — main AC-to-DC supply that feeds buck converters and fan.
- **Buck converter 6–8.4 V** — powers high-torque servos (M1, M2).
- **Buck converter 5–6.8 V** — powers mid servos (M3), motor driver and MCU (if MCU accepts this range).
- **Motor driver L298N** — powers DC gripper motor M5; note L298N has voltage drop and heat dissipation concerns so use appropriate heatsink.
- **Common ground** is mandatory between MCU, motors and drivers.

Safety and noise mitigation:

- Decoupling capacitors near servo power input (100–470 μF electrolytic + 0.1 μF ceramic).
- TVS or flyback diodes across DC motor if driver does not provide suppression.
- Separate power and signal wiring harnesses and ferrite beads to reduce EMI.

5.1.4 CAD Models Details

The mechanical design follows a **modular CAD approach** with separate parts for base_plate, shoulder_link, elbow_bracket, servo_mount, forearm_link, wrist_rotator and gripper_finger. Each part has a standardized bolt pattern for ease of assembly and replacement.

5.1.5 Material Selection & Justification

Part	**Material**	**Justification**
Frame structure / Connecting rods	Carbon-fibre tubes	Very high strength-to-weight ratio; minimal deflection for long links; lowers inertia for dynamic response.
Joint connectors / Servo mounts	Aluminium (6061-T6)	Good machinability, local stiffness at load paths, durable threaded interfaces; cost-effective for brackets.
Base plate	Steel or thick Aluminium sheet (3–5 mm)	Provides stable foundation and resistance to bending; steel if maximum rigidity required, aluminium for lighter weight.
Gripper fingers	Reinforced PLA / Nylon or Aluminium	Fingers need local high strength and wear resistance; metal for production, reinforced polymer for prototyping.
Small brackets / covers	PLA / ABS (3D printed)	Easy to prototype and iterate; replaceable if failure occurs.
Fasteners / Inserts	Stainless steel bolts, brass heat-set inserts	Corrosion resistance and durable threads for repeated assembly.

Table 5.1: Optimised material choices balancing strength, weight and manufacturability.

5.1.6 CAD Model — Part Names and Roles

CAD Part Name	Description	Connection / Range
base_plate	Foundation mounting platform; anchors whole arm	Bolted to surface; houses base servo; static.
shoulder_link	Primary arm segment	Rotatory to base via shoulder servo; transmits loads to forearm.
elbow_bracket	Intermediate joint connector	Hinged construction; allows elbow articulation.
forearm_link	Secondary arm segment	Telescoping carbon tube or rigid link to control reach.
wrist_rotator	End-effector rotation module	270° rotation range (design dependent).
servo_mount	Motor mounting brackets	Standardised mounting pattern for servos (M3/M4).
gripper_finger	End-effector fingers (x2)	Constrained open/close motion; direct interface with gripper motor.

Table 5.2: CAD parts, descriptions and connection types.

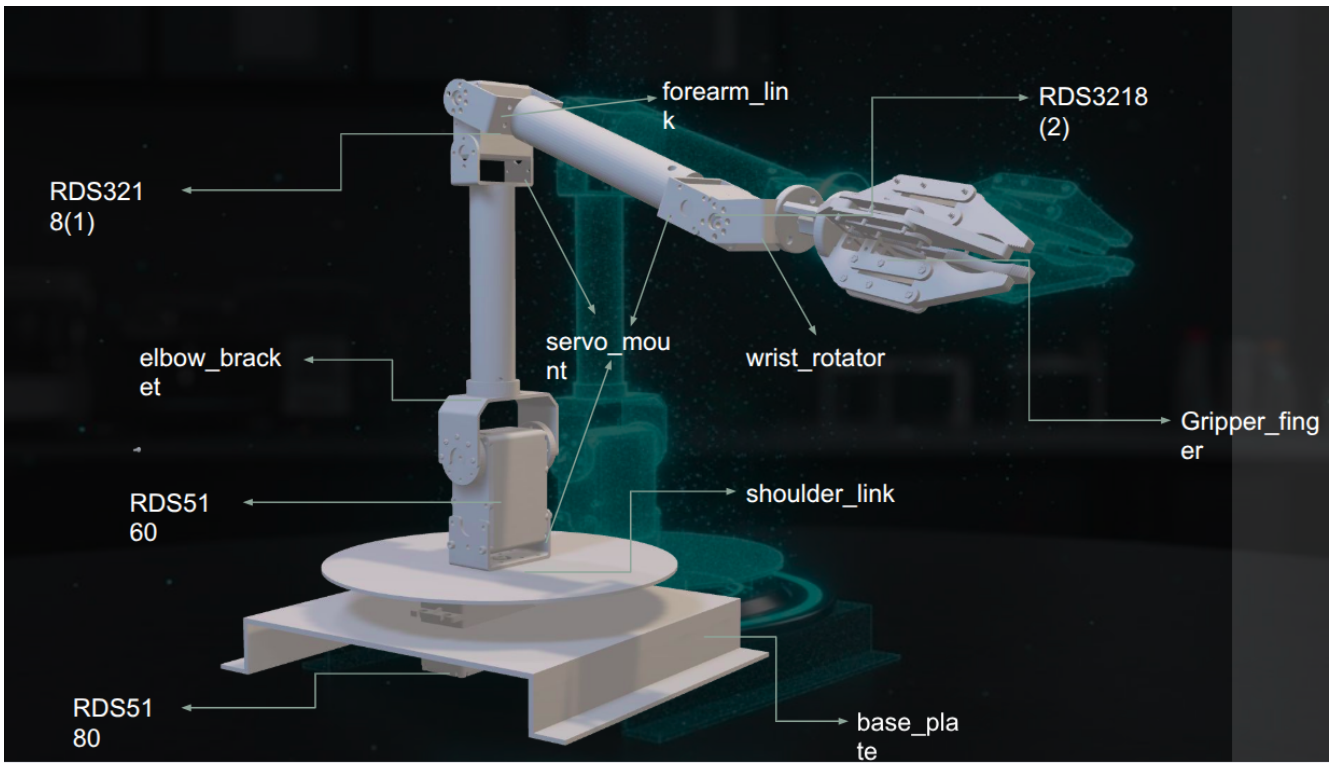

Figure 5.1: Assembly view of the modular CAD model (labels correspond to table part names).

6 Software

6.0.1 System Overview

The control pipeline is as follows:

1. A webcam captures live video feed.
2. MediaPipe Hands detects and tracks the user's left hand in real time.
3. The 3D position of the palm center is estimated using monocular depth from hand size.
4. The palm position is smoothed and mapped into a constrained spherical workspace.
5. The target position is sent to CoppeliaSim, which computes joint angles via built-in IK.
6. Joint angles are transmitted to an Arduino controlling physical servos.
7. A binary hand gesture (open palm vs. fist) controls the gripper state.

6.0.2 Hand Tracking and Palm Center Estimation

Hand tracking is performed using Google's `mediapipe.solutions.hands` with the following configuration:

- `model_complexity=0` for real-time performance
- `min_detection_confidence=0.3`, `min_tracking_confidence=0.1`

Only the **left hand** is processed (due to selfie-view camera flip). The palm center is computed as the average of five key landmarks:

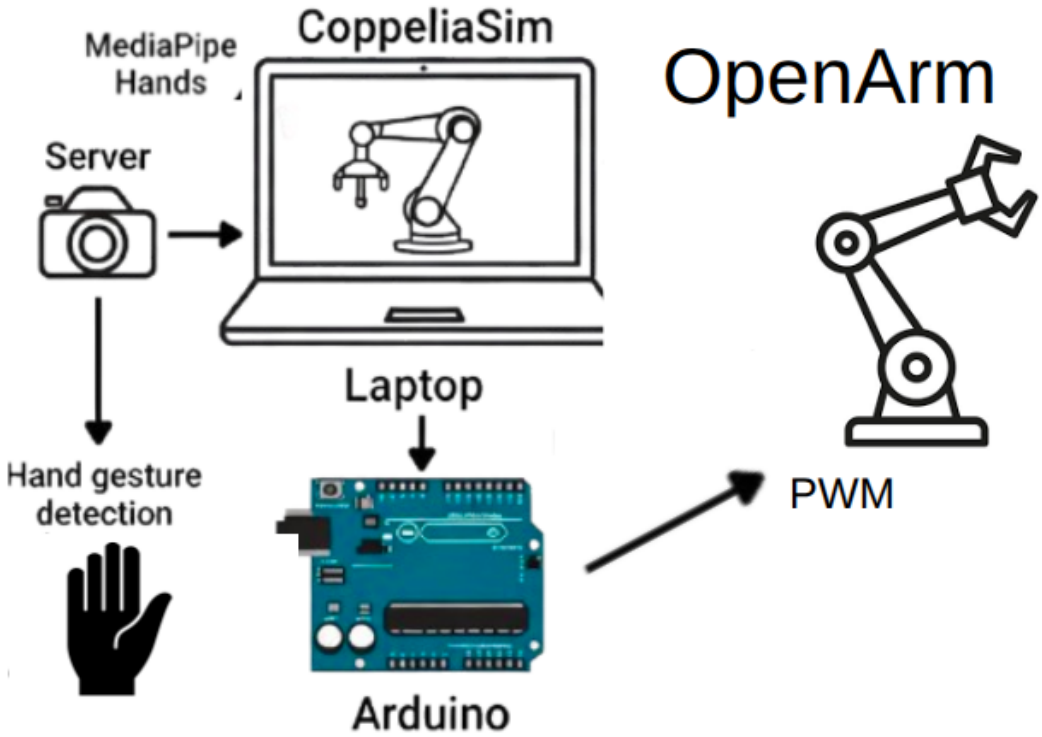

Figure 6.1: System architecture of the OpenArm gesture-controlled robotic arm.

```
PALM_LANDMARKS = [0, 5, 9, 13, 17]
```

(Wrist, index base, middle base, ring base, pinky base)

This centroid provides a stable and intuitive control point.

6.0.3 Depth Estimation via Palm Width

Monocular depth is estimated using the known average adult palm width (80 mm) between the index and pinky MCP joints (landmarks 5 and 17):

$$Z = \frac{f \times W}{w}$$

where:

- $f = 550$ pixels (empirically calibrated focal length)
- $W = 80$ mm (real-world palm width)
- w = measured pixel distance between landmarks 5 and 17

The estimated distance is clamped and normalized:

$$\text{distance} \in [50, 250] \text{ cm} \rightarrow [0.2, 0.5] \text{ (normalized forward axis)}$$

6.0.4 Workspace Mapping and Constraints

The normalized hand coordinates are mapped as:

$$x = 0.5 - \frac{c_x}{w} \qquad \text{(lateral: left/right)} \tag{6.1}$$

$$y = 0.5 - \frac{c_y}{2h} \qquad \text{(vertical: up/down)} \tag{6.2}$$

$$z = 0.5 - \frac{d - 50}{200} \qquad \text{(depth: near/far)} \tag{6.3}$$

These are clamped to a spherical workspace centered at `SPHERE_CENTER = (0.011, -0.00183, 0.131)` with radius 0.5 m, and $z \geq 0$.

Exponential moving average smoothing ($\alpha = 0.05$) is applied to reduce jitter.

6.0.5 Gripper Control via Gesture Recognition

Gripper state is determined by comparing the y-coordinates of the middle finger tip (landmark 12) and its base (landmark 9):

```
if index_tip.y < index_base.y:
    palm_open = True   # Hand open     gripper open
    motor_dir = 1
else:
    palm_open = False  # Fist     gripper closed
    motor_dir = 2
```

The command `MD 1` or `MD 2` is sent to the Arduino to control gripper direction.

6.0.6 Communication Protocol

Two types of serial commands are sent to the Arduino:

- `S A B C D` – Set servo angles (base, shoulder, elbow, wrist) in degrees
- `MD X` – Set gripper motor direction (`1`=open, `2`=close)

Angles are adjusted with hardware-specific offsets and scaling (e.g., elbow and wrist use 180°/270° mapping).

6.0.7 Integration with CoppeliaSim

The system uses the `coppeliasim-zmqremoteapi` to:

- Set a dummy target object position in the simulated arm workspace
- Read resulting joint angles via CoppeliaSim's internal IK solver
- Step the simulation synchronously

This enables safe testing and tuning before deployment on real hardware.

6.0.8 User Interaction

- Press `q` to toggle control `ENABLED/DISABLED`
- Press `ESC` to exit gracefully
- Real-time overlay shows palm center, normalized coordinates, and gripper state

6.0.9 Key Features

Feature	**Implementation**
Real-time hand tracking	MediaPipe Hands
3D position estimation	Monocular depth from hand size
Smooth, stable control	Exponential moving average filtering
Constrained workspace	Spherical bounding + Z0
Gesture-based gripper	Open palm vs. fist detection
Simulation-to-reality bridge	CoppeliaSim + ZMQ API
Physical control	Arduino + servo drivers

Table 6.1: Core features of the OpenArm system.

7 Conclusion

The OpenArm system demonstrates a complete, functional pipeline for intuitive, vision-based teleoperation of a robotic arm using only a webcam and natural hand movements. The combination of robust hand tracking, physics-aware constraints, and real-time gesture recognition enables fluid and expressive control suitable for education, assistive technology, and human–robot interaction research.